

深圳大学实验报告

课程名称： 算法设计与分析

实验项目名称： 实验五 桥

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 杨烜

报告人： 姚泽棉 学号： 2024150026 班级： 高性能班

实验时间： 2026/05/29

实验报告提交时间： 2026/06/08

实验五 桥

一、实验目的：

- (1) 掌握图的连通性。
- (2) 掌握并查集的基本原理和应用。

二、内容：

1. 桥的定义

在图论中，一条边被称为“桥”代表这条边一旦被删除，这张图的连通块数量会增加。等价地说，一条边是一座桥当且仅当这条边不在任何环上。一张图可以有零或多座桥。

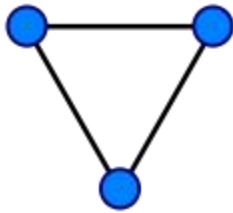


图 1 没有桥的无向连通图

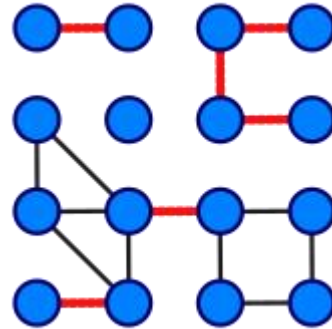


图 2 这是有 16 个顶点和 6 个桥的图
(桥以红色线段标示)

2. 求解问题

找出一个无向图中所有的桥。

3. 算法

(1) 基准算法

For every edge (u, v) , do following

- a) Remove (u, v) from graph
- b) See if the graph remains connected (We can either use BFS or DFS)
- c) Add (u, v) back to the graph.

(2) 应用并查集设计一个比基准算法更高效的算法。不要使用 Tarjan 算法，如果使用 Tarjan 算法，仍然需要利用并查集设计一个比基准算法更高效的算法。

三、实验要求

1. 实现上述基准算法。
2. 设计的高效算法中必须使用并查集，如有需要，可以配合使用其他任何数据结构。
3. 用图 2 的例子验证算法正确性。
4. 使用文件 `mediumG.txt` 和 `largeG.txt` 中的无向图测试基准算法和高效算法的性能，记录两个算法的运行时间。
5. 设计的高效算法的运行时间作为评分标准之一。
6. 提交程序源代码。
7. 实验报告中要详细描述算法设计的思想，核心步骤，使用的数据结构。

四、实验过程及结果

一：基准法求解

1. 算法原理

- (1) 由桥的定义可知，删除桥之后，图的连通分支数量就会增加。所有只需要比较删除前后连通分支数量即可判断该边是否为桥。
- (2) 利用 DFS 算法计算连通分支数量。
- (3) 计算连通分支数量方法：
 - ① 首先创建长度大小等于节点个数的访问数组，并对每个元素初始化为 `false`，然后对每个节点进行以下操作。
 - ② 若当前节点已经被访问过，则遍历下一个节点
 - ③ 若当前节点没被访问，连通分量个数加 1，同时对该节点进行 DFS 遍历，将遍历过程中的点对应访问元素的值设置成 1

2. 伪代码

BASIC:

```
A = Count_connected_components
For every edge (u, v):
    Remove (u, v) from graph
    B = Count_connected_components
    If B > A
        (u,v) is a bridge
    Add (u, v) back to the graph.
```

Count_connected_components:

```
Count = 0
For every vertex (v):
    If visited[v] = false:
        DFS (v)
    Count++
return Count
```

DFS(v):

```
visited[v] = true
for every adjacent points (next) of (v):
    if visited[next] = false:
        DFS (next)
```

3. 时间复杂度分析

对全图做 DFS 需要 $O(n + e)$

有 e 条边，需要对全图做 DFS，总时间复杂度为 $O(en + e^2)$

对于稀疏图 $e = n$ ，对于稠密图 $e = n^2$

稀疏图： $O(n^2)$ ，稠密图： $O(n^4)$

4. 优化模型

(1) 算法原理：

① 在基准法的基础上，删除边后不需要对全图进行 DFS。

② 对删除边的其中一个结点做 DFS，如果在深度优先搜索过程中，搜索到另一结点则说明在同一连通分支，提前结束搜索。

(2) 伪代码：

IMPROVED_BASIC:

For every edge (u, v) :

Remove (u, v) from graph

DFS (u)

If v is visited during DFS

(u, v) is a bridge

Add (u, v) back to the graph.

(3) 时间复杂度分析：

有 e 条边，需要做 1 次 DFS，总时间复杂度上限为 $O(en + e^2)$

对于稀疏图 $e = n$ ，对于稠密图 $e = n^2$

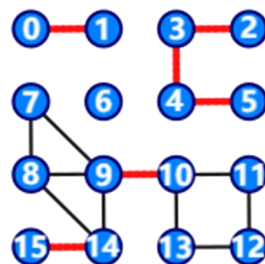
稀疏图： $O(n^2)$ ，稠密图： $O(n^4)$

(4) 优化分析：

图越稀疏时，连通分支增多，优化后只需要对一个连通分支进行 DFS，可以极大程度上增快搜索速度。

5. 正确性验证

以下图为输入，验证算法是否正确。



输出结果如下图，算法的正确性得以验证。

[基准算法结果]

0 1

2 3

3 4

4 5

9 10

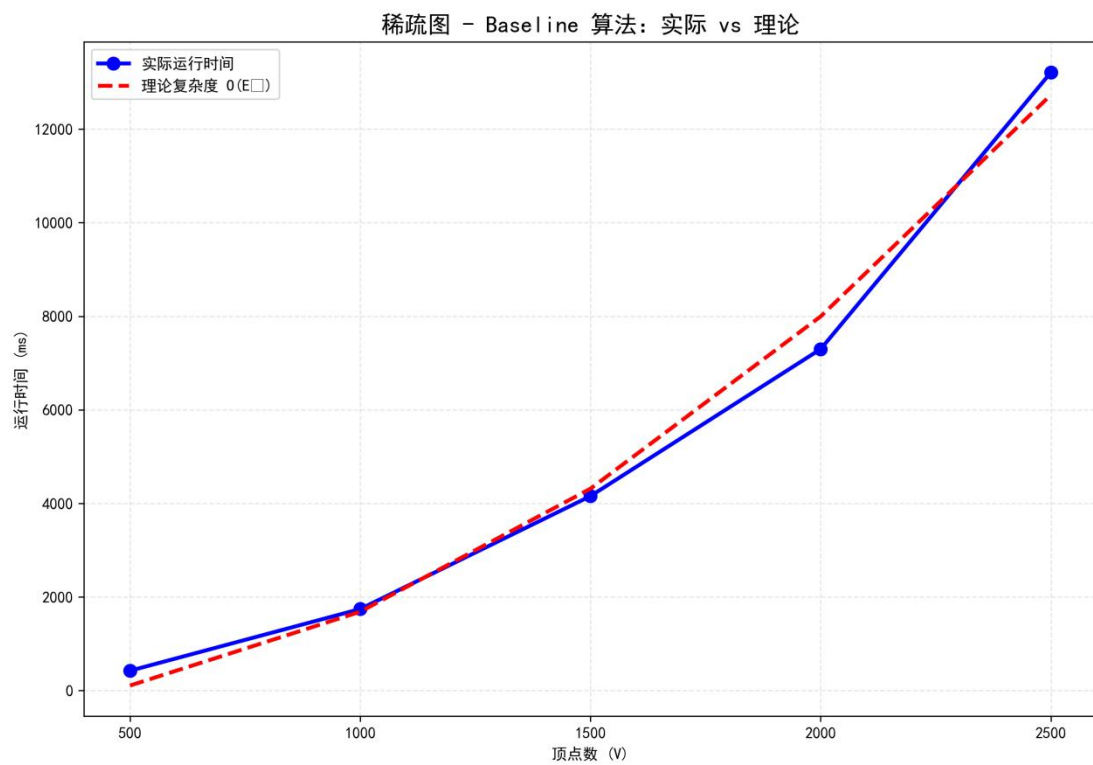
14 15

桥的数量: 6

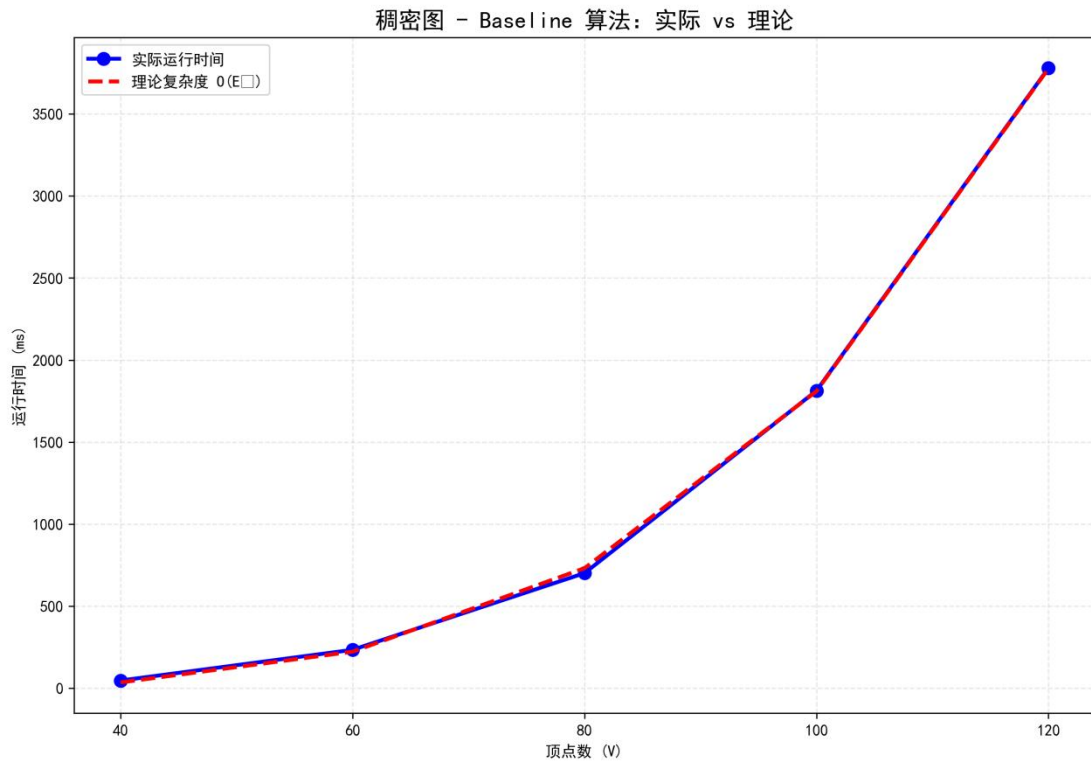
运行时间: 0.1246 ms

6. 数据分析

稀疏图: $e = n$ 时间复杂度: $O(n^2)$



稠密图: $e = n^2$ 时间复杂度: $O(n^4)$

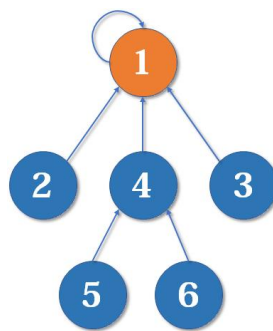


二：并查集求解

*并查集

A. 并查集原理

- ① 并查集的重要思想在于，用集合中的一个元素代表集合。
- ② 这是一个树状的结构，要寻找集合的代表元素，只需要一层一层往上访问父节点（图中箭头所指的圆），直达树的根节点（图中橙色的圆）即可。根节点的父节点是它自己。我们可以直接把它画成一棵树



B. 伪代码

初始化：

Init()

```
for i = 0 to n
    parent[i] = i;
```

查询

Get_Parent(x)

```
if(parent[x] == x)
    return x;
else
    return Get_Parent(parent[x]);
```

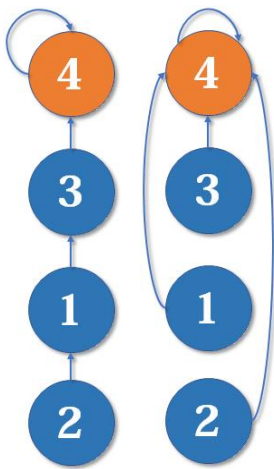
合并

Union(i, j)

```
parent[Get_Parent(i)] = Get_Parent(j);
```

C. 优化方案

a. 路径压缩



随着链越来越长，我们想要从底部找到根节点会变得越来越难。可以使用路径压缩的方法。既然只关心一个元素对应的根节点，那就将每个元素直接指向根节点。

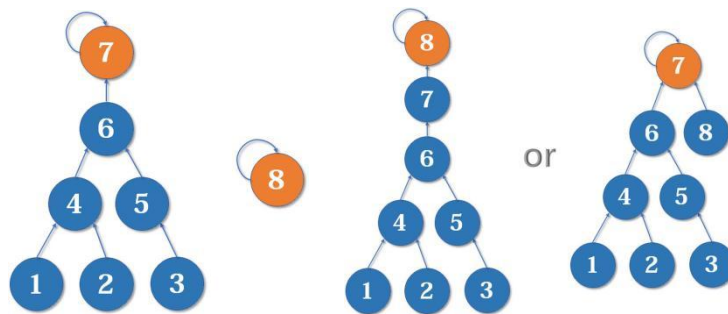
实现的方式是在查询的过程中，把沿途的每个节点的父节点都设为根节点即可。下一次再查询时，就可以省很多事。

Get_Parent(x)

```
If (x == parent[x])
    return x;
else
    parent[x] = Get_Parent(parent[x]); // 父节点设为根节点
    return parent[x]; // 返回父节点
```

b. 平衡处理

由于路径压缩只在查询时进行，也只压缩一条路径，所以并查集最终的结构仍然可能是比较复杂的。



显然应该把简单的树往复杂的树上合并，而不是相反。因为这样合并后，到根节点距离变长的节点个数比较少。

实现方法是用一个数组 `rank[]` 记录每个根节点对应的树的深度（如果不是根节点，其 `rank` 相当于以它作为根节点的子树的深度）。一开始，把所有元素的 `rank` 设为 1。合并时比较两个根节点，把 `rank` 较小者往较大者上合并。

Union(i, j)

```
x = Get_Parent(i), y = Get_Parent(j);    //先找到两个根节点
if (rank[x] <= rank[y])
    parent[x] = y;
else
    parent[y] = x;
if (rank[x] == rank[y] && x != y)
    rank[y]++;    //如果深度相同且根节点不同,则新的根节点的深度+1
```

伪代码

BASIC_Union:

A = Count_connected_components

$$\mathbf{B} = \mathbf{v}$$

For every edge (e_1, e_2) :

Remove (e_1, e_2) from graph

For every edge (e_3, e_4) :

Union (e_3, e_4)

```

# if parent[e3] = parent[e4], B = B - 1

```

If $B > A$

(e_1, e_2) is a bridge

Add (e_1, e_2) back to the graph.

1. 算法原理

(1) 读入图



(2) 建立生成森林

利用 BFS 遍历图。将第一次访问到某节点所经过的边加入生成森林。生成森林中的边称为树边。



(3) 找非树边

对于遍历过程中遇到但不属于生成森林的边： (u, v) 称为非树边。非树边与生成森林中的路径共同构成一个环。



(4) 标记环边

对于每条非树边: $u \text{-----} v$ 从 u 和 v 分别向父节点方向回溯, 直到二者相遇。相遇点即两点在生成森林中的公共祖先。回溯路径上的所有边均属于环。因此这些边不可能是桥



(5) 并查集路径压缩优化

↓

(6) 输出桥

2. 复杂度分析

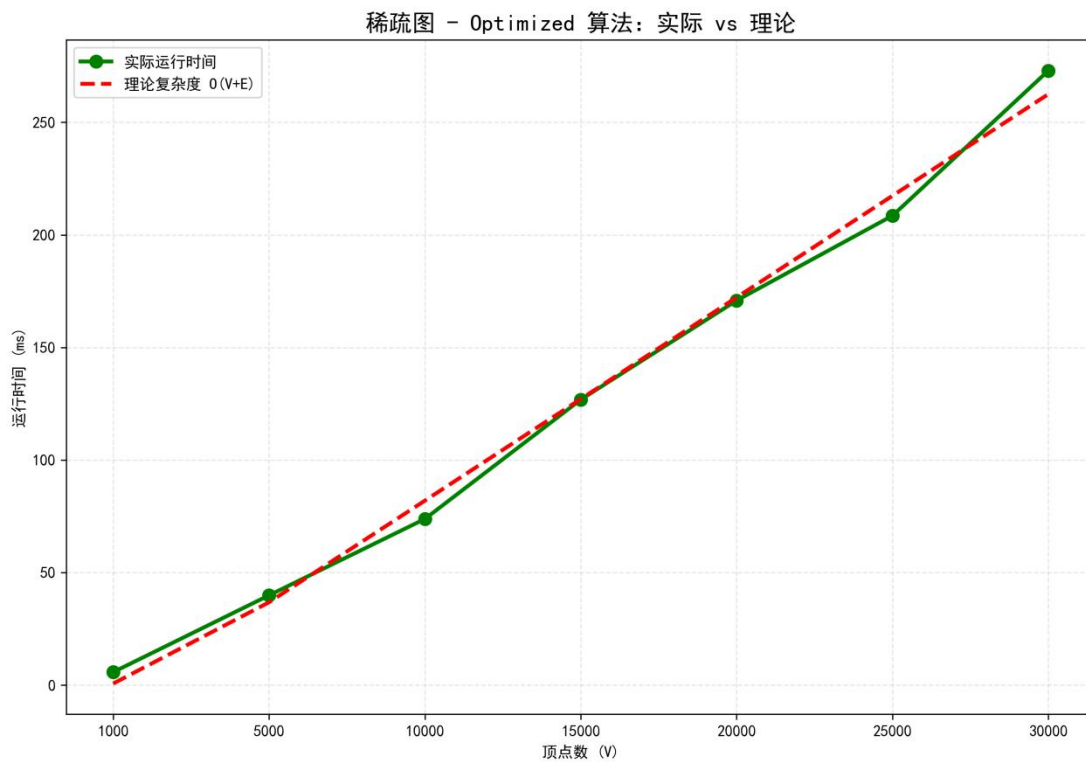
通常直接写 $O(n+e)$

4、正确性验证

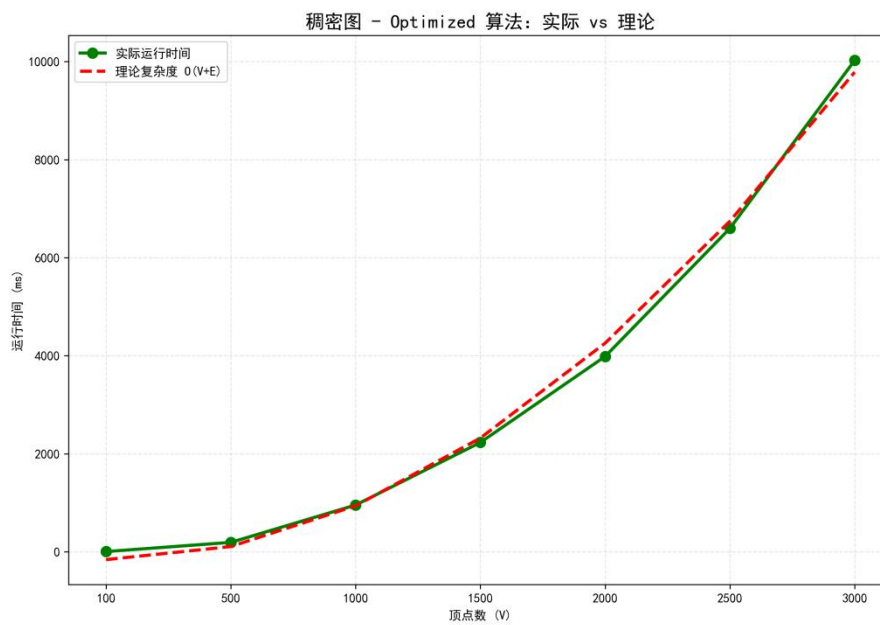
【并查集优化算法结果】

运行时间: 0.0577 ms

稀疏图: $e = n$ 时间复杂度: $O(n)$



稠密图: $e = n^2$ 时间复杂度: $O(n^2)$



三、各算法在 mediumDG、largeG 中的表现
mediumDG:

```
===== mediumDG =====
```

```
【基准算法结果】
```

```
桥的数量：0
```

```
运行时间：6.9849 ms
```

```
【并查集优化算法结果】
```

```
桥的数量：0
```

```
运行时间：0.4621 ms
```

$6.98 / 0.46 \approx 15.1$ 倍

largeG:

```
===== largeG =====
```

```
【基准算法】跳过 - 边数过多，时间复杂度过高  $O(E^2)$ 
```

```
【并查集优化算法结果】
```

```
95760 773903
```

```
148837 372243
```

```
317390 589095
```

```
461822 548437
```

```
467595 630627
```

```
467595 907820
```

```
639238 969090
```

```
658123 724640
```

```
桥的数量：8
```

```
运行时间：33489.9518 ms
```

总结：

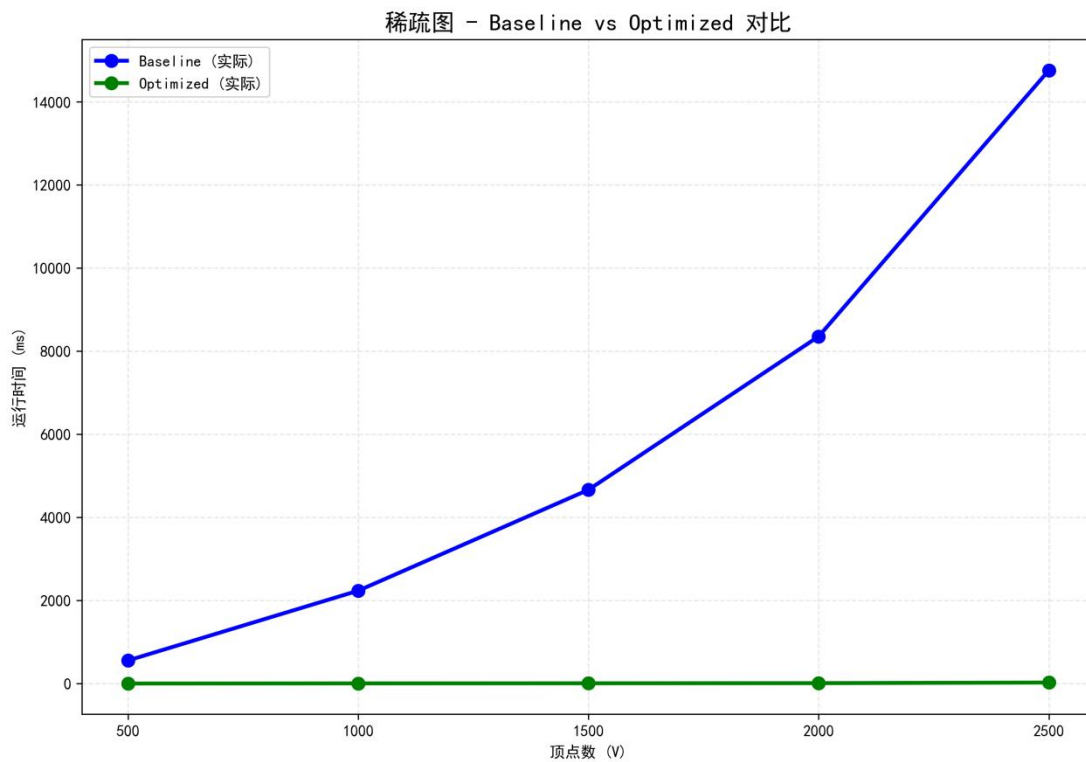
在 mediumDG 数据集上，两种算法均能够快速完成桥的查找。

其中：Baseline 算法耗时 6.98ms，Optimized 算法耗时 0.46ms，性能提升约 15 倍。

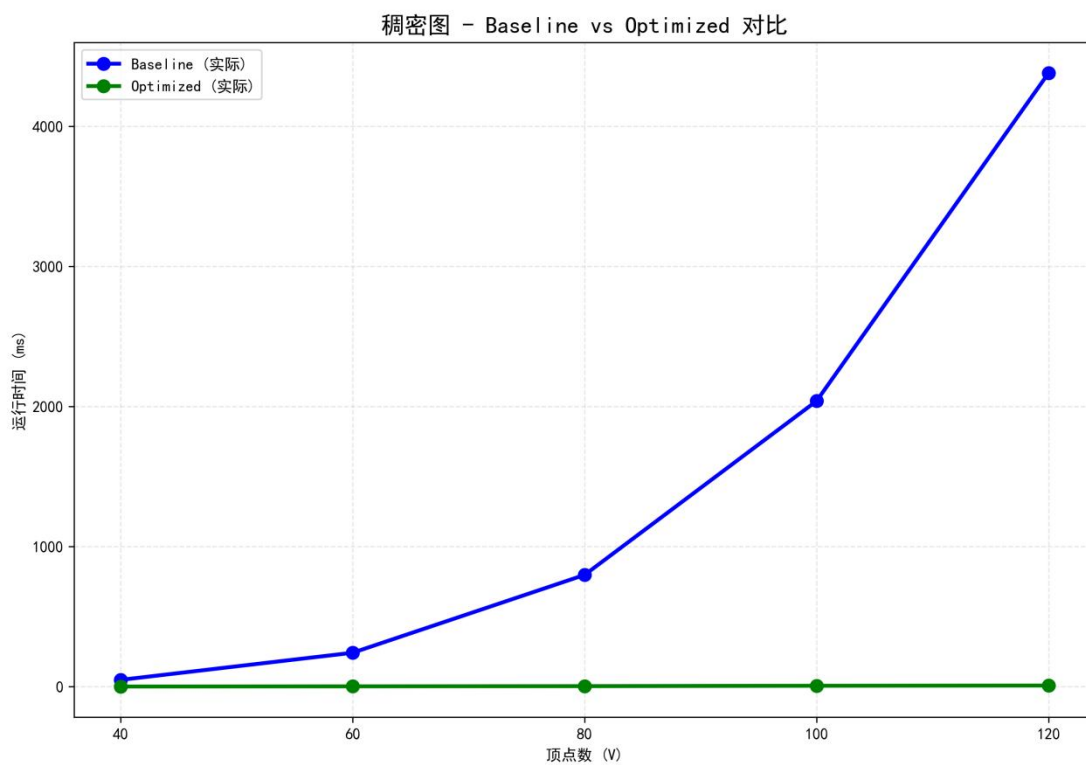
在 largeG 数据集上，由于边数规模显著增加，Baseline 算法运行时间过长，因此未完成测试。

Optimized 算法仍能在 33.49ms 内完成桥的查找，体现出较好的扩展性。

稀疏图对比：



稠密图对比:



五、经验总结

本实验围绕无向图中的桥展开。基准算法直接按照定义，通过逐条删除边并判断端点连通性来识别桥，思想清晰但效率较低。高效算法利用“桥不在任何环上”的等价性质，先用并查集构造生成森林，再通过非树边标记环上的所有边，最后剩余未被标记的树边即为桥。

通过实验发现，桥与环之间存在天然联系。如果一条边位于某个环中，即使删除该边，图中仍然存在替代路径，因此不可能成为桥。高效算法正是利用这一性质，避免了对每条边重复进行图搜索，从而显著提升了运行效率。

同时实验表明,并查集不仅可以维护连通分量,还可以配合路径压缩减少重复路径扫描,使算法能够处理大规模图数据。通过本实验,我进一步理解了连通性、生成树、环与桥之间的关系,也加深了对并查集在图算法中应用价值的认识。

指导教师批阅意见：	
成绩评定：	指导教师签字： 年 月 日
备注：	

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。